

# Large Language Diffusion Models

Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, Chongxuan Li

Presenter: Ye YUAN





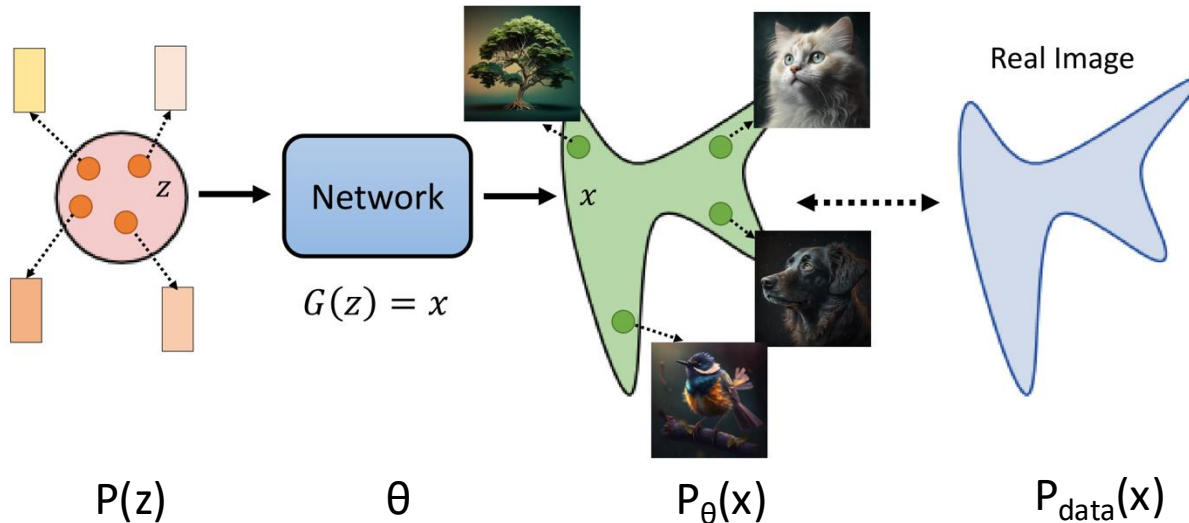
# Contents

---

- Preliminary Knowledge
- Two Paradigms of Diffusion Models for Language Modeling
- Large Language Diffusion Models
- Discussion and Conclusions
- Questions and Answers



# Preliminary – Likelihood-based Generative Models



Sample  $\{x^1, x^2, \dots, x^m\}$  from  $P_{data}(x)$

$$\begin{aligned}
 \theta^* &= \arg \max_{\theta} \prod_{i=1}^m P_{\theta}(x^i) = \arg \max_{\theta} \log \prod_{i=1}^m P_{\theta}(x^i) \\
 &= \arg \max_{\theta} \sum_{i=1}^m \log P_{\theta}(x^i) \approx \arg \max_{\theta} E_{x \sim P_{data}} [\log P_{\theta}(x)] \\
 &= \arg \max_{\theta} \int_x P_{data}(x) \log P_{\theta}(x) dx - \int_x P_{data}(x) \log P_{data}(x) dx \quad (\text{not related to } \theta) \\
 &= \arg \max_{\theta} \int_x P_{data}(x) \log \frac{P_{\theta}(x)}{P_{data}(x)} dx = \arg \min_{\theta} KL(P_{data} || P_{\theta}) \quad \text{Difference between } P_{data} \text{ and } P_{\theta}
 \end{aligned}$$

Maximum Likelihood = Minimize KL Divergence

- $P_{data}(x)$ : probability distribution of the data
- $P_{\theta}(x)$ : approximate probability distribution of the data
- $P(z)$ : probability distribution of the latent variable, usually a Gaussian distribution



# Preliminary – Evidence Lower Bound (ELBO)

- In practice, we usually maximize the log likelihood.
- Sometimes, the log likelihood is also called evidence.
- We can maximize the ELBO instead.

$$\begin{aligned}
 \log p(\mathbf{x}) &= \log p(\mathbf{x}) \int q_{\phi}(\mathbf{z}|\mathbf{x}) d\mathbf{z} \\
 &= \int q_{\phi}(\mathbf{z}|\mathbf{x}) (\log p(\mathbf{x})) d\mathbf{z} \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x})] \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p(\mathbf{x}, \mathbf{z}) q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x}) q_{\phi}(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{q_{\phi}(\mathbf{z}|\mathbf{x})}{p(\mathbf{z}|\mathbf{x})} \right] \\
 &= \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right] + D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}|\mathbf{x})) \\
 &\geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[ \log \frac{p(\mathbf{x}, \mathbf{z})}{q_{\phi}(\mathbf{z}|\mathbf{x})} \right].
 \end{aligned}$$

$$p(\mathbf{x}) = \frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{z}|\mathbf{x})}$$

Evidence Lower Bound (ELBO)

# Preliminary – Denoising Diffusion Models

- Diffusion Process (Forward Process)



- Denoising Process (Backward Process)





# Preliminary – ELBO of Diffusion Models

$\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]$ : a reconstruction term, predicting the log probability of the original data sample given the first-step latent.

$D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T))$ : represents how close the distribution of the final noisified input is to the standard Gaussian prior.

$\sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))]$ : a denoising matching term. The  $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$  defines how to denoise a noisy image  $\mathbf{x}_t$  with access to what the final, completely denoised image  $\mathbf{x}_0$  should be.

$$\begin{aligned} \log p(\mathbf{x}) &= \log \int p(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \\ &\geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[ \log \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \\ &= \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)] - D_{KL}(q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)) \\ &\quad - \sum_{t=2}^T \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{KL}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))] . \end{aligned}$$



$\mathbf{x}_t$



$\mathbf{x}_{t-1}$



$\mathbf{x}_0$

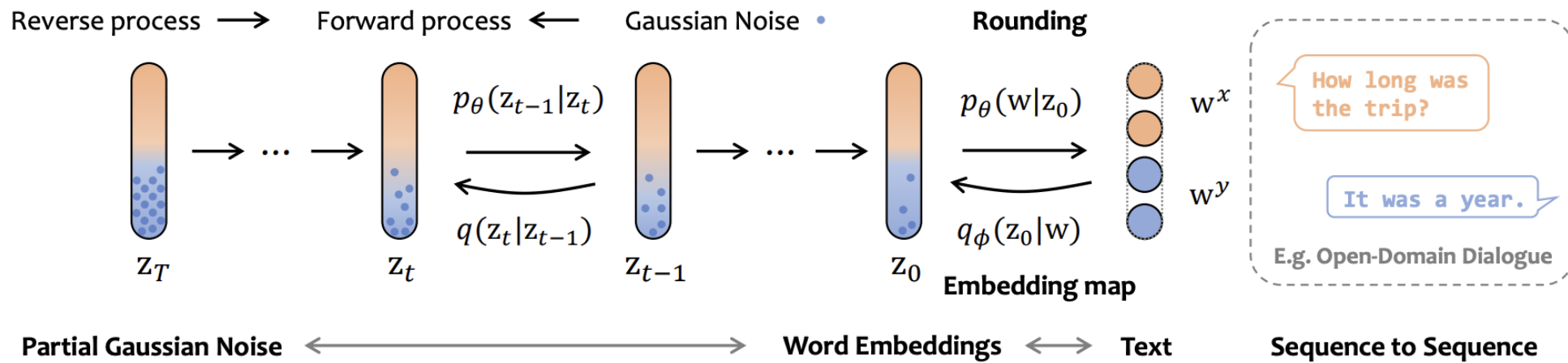
[1] Luo, C. Understanding diffusion models: A unified perspective, 2022.

[2] Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (2020).



# Diffusion for Language Modeling (Continuous)

- Convert to Continuous State Space:



# Diffusion for Language Modeling (Discrete)

- Reformulate Two Key Equations in Discrete State Space:

Suppose we have a vocabulary table of size  $K = 4$ : “apple”, “banana”, “carrot”, “[MASK]”.

Define a transition matrix  $Q_t \in \mathbb{R}^{K \times K}$  at time  $t$  that govern the corruption process:

$$Q_t = \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0.7 \\ 0.1 & 0.1 & 0.1 & 0.7 \\ 0.1 & 0.1 & 0.1 & 0.7 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

This means any normal token (e.g., “banana”) has a 70% chance of turning into [MASK] at time  $t$ . Once a token becomes [MASK], it stays that way.





# Diffusion for Language Modeling (Discrete)

“apple”, “banana”, “carrot”, “[MASK]”.

- Forward Process:

$$q(x_t | x_{t-1}) = \text{Cat}(x_t; p = x_{t-1} Q_t)$$

$$Q_t = \begin{bmatrix} 0.1 & 0.1 & 0.1 & 0.7 \\ 0.1 & 0.1 & 0.1 & 0.7 \\ 0.1 & 0.1 & 0.1 & 0.7 \\ 0.0 & 0.0 & 0.0 & 1.0 \end{bmatrix}$$

$x_{t-1}$  is a one-hot row vector. For “banana”,  $x_{t-1} = [0, 1, 0, 0]$ .

$x_{t-1} Q_t = [0.1, 0.1, 0.1, 0.7]$ : give the probability of transitioning to other tokens.

- $q(x_t | x_0) = \text{Cat}(x_t; p = x_0 \bar{Q}_t)$ , with  $\bar{Q}_t = Q_1 Q_2 \dots Q_t$

- $q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0) q(x_{t-1} | x_0)}{q(x_t | x_0)} = \text{Cat}\left(x_{t-1}; p = \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top}\right)$

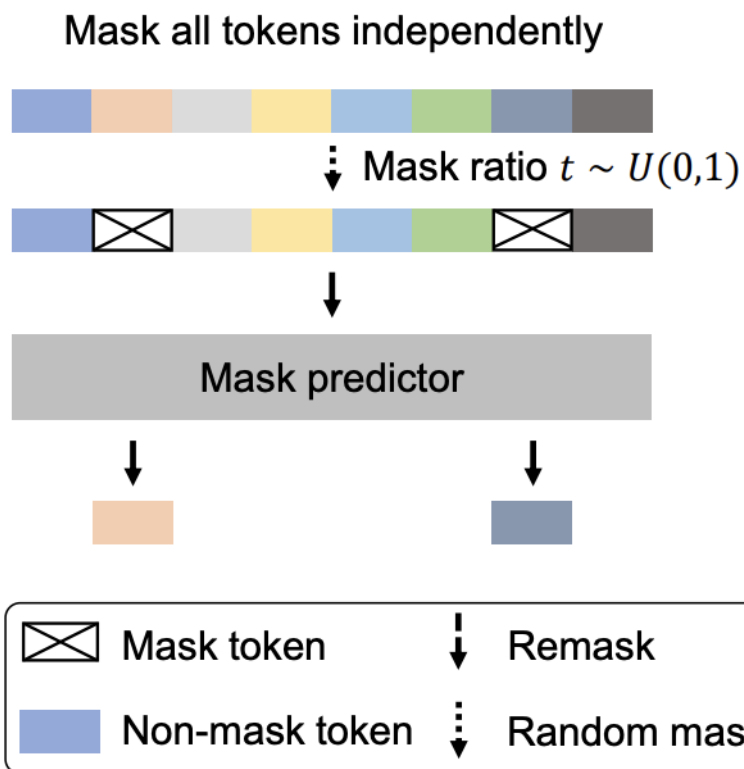
how likely each candidate  $x_{t-1}$  would result in the current  $x_t$

how likely  $x_t$  was generated from the original input  $x_0$

how likely each  $x_{t-1}$  was generated from the original input  $x_0$

# Large Language Diffusion Models - Pretraining

- This paper introduces large language diffusion model (LLaDA), with discrete states formulation. Each token can only be corrupted to [MASK] token in this paper.



$$x_0 \sim p_{\text{data}}, t \sim U(0, 1]$$

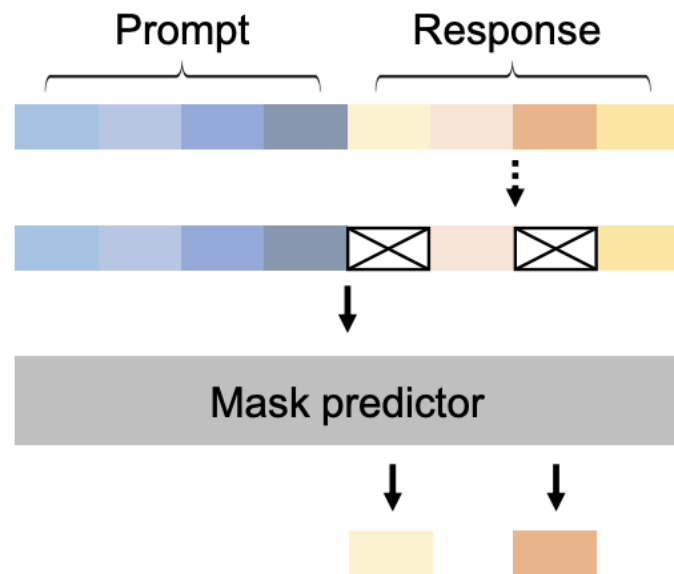
$$x_t \sim q_{t|0}(x_t|x_0)$$

This is like the BERT pretraining phase but with random mask ratio between 0 to 1.



# Large Language Diffusion Models – SFT

- To enhance the instruction following capability of LLaDA, the authors perform supervised fine-tuning, similar to other autoregressive LLMs like Llama.



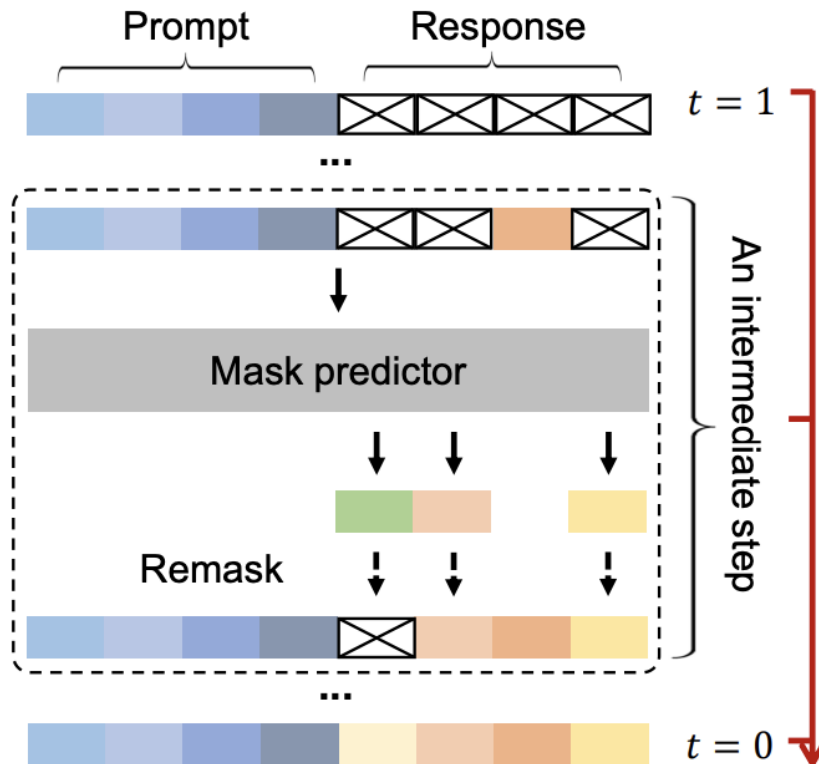
$$p_0, r_0 \sim p_{\text{data}}, t \sim \text{U}(0, 1]$$
$$r_t \sim q_{t|0}(r_t|r_0)$$

$p_0$  is a prompt,  $r_0$  is a response.  
Only apply masking to response.



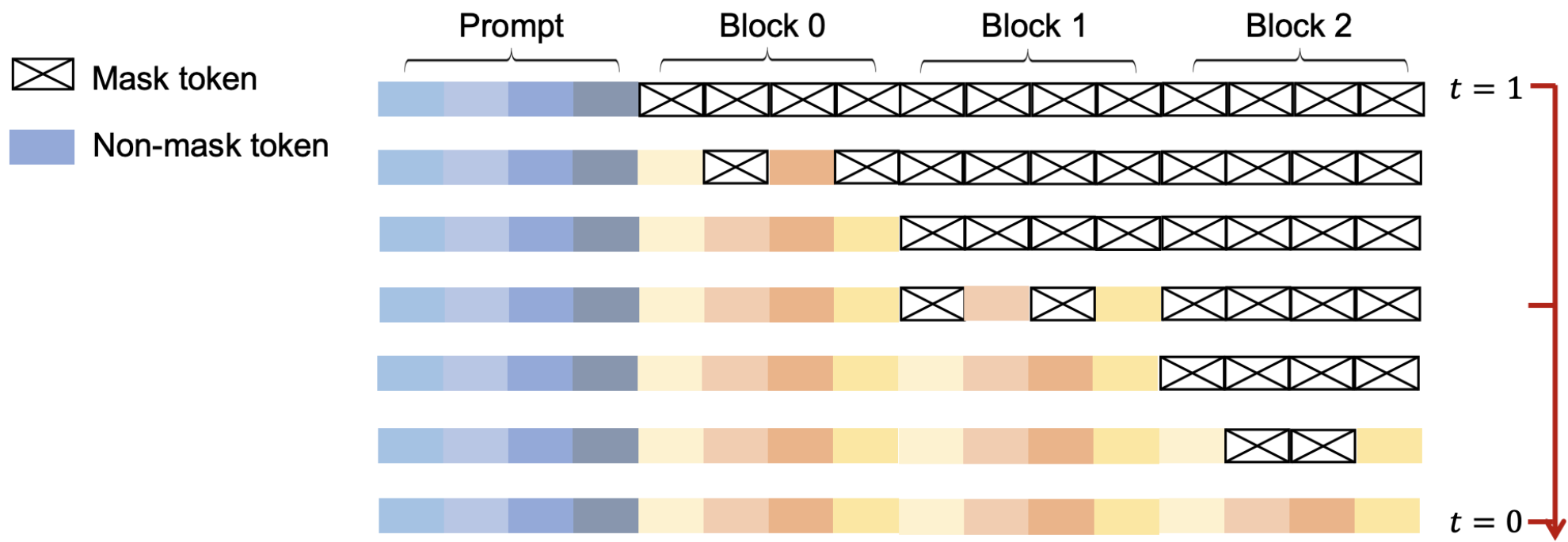
# Large Language Diffusion Models - Sampling

- LLaDA predicts all [MASK] tokens at once, then remasks some tokens and predicts them again. By doing this, the model can iteratively generate a response.



- Initialize the response as a sequence of [MASK] tokens with length  $s$ . The total steps for generation is  $T$ . The time steps could be  $[0, 1/T, \dots, 1]$ , and we remask  $s \times i/T$  tokens at discrete time step  $i$ .
- Three different remask strategies:
  - Random**: randomly remask tokens.
  - Low-Confidence**: remask lowest confidence tokens.
  - Semi-Autoregressive**: divide the sequence into several blocks and generate them from left to right.

# Semi-Autoregressive Remasking



# Discussion and Conclusion

- Advantages of LLaDA:

- Faster sampling than autoregressive model.
- Better reversal reasoning ability.

*Table 3. Comparison in the Poem Completion Task.*

|                     | Forward     | Reversal    |
|---------------------|-------------|-------------|
| GPT-4o (2024-08-06) | <b>82.7</b> | 34.3        |
| Qwen2.5 7B Instruct | 75.9        | 38.0        |
| LLaDA 8B Instruct   | 48.8        | <b>42.4</b> |

- Disadvantages of LLaDA:

- Harder to calculate the probability of a generated sequence. (Need Monte-Carlo Estimations)
- Generated sequence length and total generation steps are hyperparameters.
- Still lower performance on general tasks than autoregressive LLMs.

# Thanks for your attention!



# References

---

- [1] Luo, C. Understanding diffusion models: A unified perspective, 2022.
- [2] Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In Advances in Neural Information Processing Systems (2020).
- [3] S. Gong, M. Li, J. Feng, Z. Wu, and L. Kong, DiffuSeq: Sequence to Sequence Text Generation with Diffusion Models. International Conference on Learning Representations (2023).
- [4] J. Austin, D. D. Johnson, J. Ho, D. Tarlow, and van, Structured Denoising Diffusion Models in Discrete State-Spaces. In Advances in Neural Information Processing Systems (2021).
- [5] S. Nie et al., “Large Language Diffusion Models,” 2025.



# Recent Progress

---

- I'm leading four projects recently as a core contributor:
  - RBC Borealis Internship:
    - Embedding-Based Contextual-Aware Reranker (Put on arXiv after patent, target ICLR)
    - Auto Template Generation for Table Summarization (ICLR workshop or ACL ARR short paper)
  - MSR-Mila Collaboration (Generative Models for Entity Linking) (Get an arXiv asap, may target ICLR or journals like JMLR/TMLR.)
  - Diffusion Language Model for Offline Model-based Optimization (Get an arXiv asap, target ICLR)
- I'm actively participating other ongoing projects within our group:
  - LLMs for Personalization: RAG LLM, Steering LLM Generation
  - Samsung-McGill Collaboration: Multi-LLMs system and debating

# Recent Progress - EBCAR

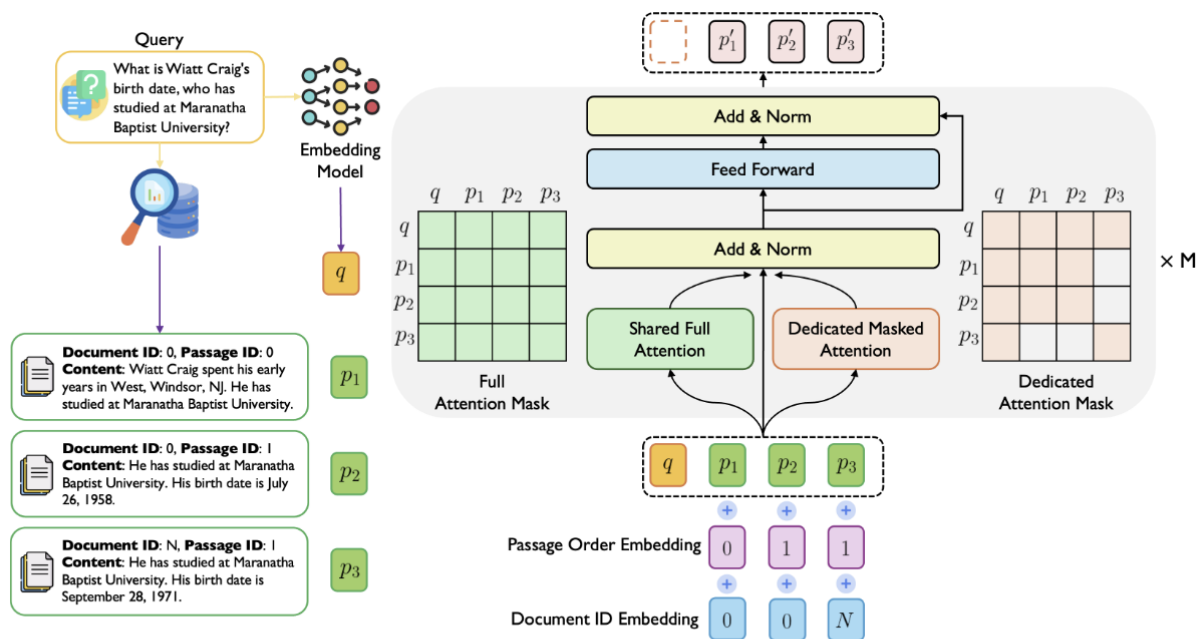


Figure 2: Overview of the EBCAR architecture. Candidate passage embeddings are enriched with document ID and passage position information, then processed jointly with the query embedding through a Transformer encoder. The hybrid attention mechanism combines a shared full attention module and a document-aware masked attention module.

| Method              | Training     | Size | MLDR  | SQuAD | NarrQA | COVID | ESG   | Football | Geog  | Insur | Avg   | Throughput |
|---------------------|--------------|------|-------|-------|--------|-------|-------|----------|-------|-------|-------|------------|
| BM25                | -            | -    | 65.42 | 45.07 | 36.20  | 39.79 | 10.10 | 4.69     | 23.66 | 0.00  | 28.12 | 263.16     |
| Contriever          | -            | -    | 60.23 | 54.63 | 66.97  | 31.02 | 15.69 | 5.95     | 46.39 | 2.75  | 35.45 | 29.67      |
| monoBERT (base)     | ConTEB Train | 110M | 40.44 | 27.94 | 36.63  | 20.52 | 11.67 | 4.01     | 29.23 | 2.92  | 21.67 | 4.24       |
| monoT5 (base)       | ConTEB Train | 223M | 26.76 | 19.75 | 41.21  | 24.56 | 10.37 | 5.69     | 36.70 | 4.40  | 21.18 | 1.69       |
| monoT5 (base)       | MS-MARCO     | 223M | 70.47 | 67.00 | 58.89  | 41.51 | 18.61 | 9.68     | 54.47 | 2.20  | 40.37 | 3.61       |
| duoT5 (base)        | MS-MARCO     | 223M | 42.34 | 68.63 | 21.46  | 18.19 | 12.69 | 8.84     | 47.18 | 2.57  | 27.74 | 0.18       |
| RankVicuna          | MS-MARCO     | 7B   | 79.30 | 66.59 | 79.48  | 51.87 | 22.97 | 11.46    | 71.08 | 4.27  | 48.38 | 0.17       |
| RankZephyr          | MS-MARCO     | 7B   | 82.34 | 69.06 | 81.18  | 53.15 | 26.42 | 11.63    | 72.91 | 3.51  | 50.03 | 0.17       |
| FIRST               | MS-MARCO     | 7B   | 74.78 | 61.55 | 78.62  | 41.17 | 20.06 | 7.88     | 60.92 | 3.21  | 43.52 | 1.73       |
| FirstMistral        | MS-MARCO     | 7B   | 74.41 | 62.19 | 78.69  | 41.35 | 20.26 | 7.78     | 61.79 | 3.40  | 43.73 | 1.78       |
| LiT5Distil          | MS-MARCO     | 248M | 60.33 | 54.87 | 66.94  | 31.22 | 16.07 | 5.98     | 46.66 | 2.78  | 35.61 | 0.66       |
| PE-Rank             | MS-MARCO     | 8B   | 52.58 | 45.00 | 51.58  | 33.40 | 19.92 | 6.29     | 46.25 | 1.37  | 32.05 | 0.81       |
| RankGPT (Mistral)   | -            | 7B   | 65.75 | 56.61 | 68.29  | 33.23 | 15.04 | 7.67     | 51.75 | 3.05  | 37.67 | 0.12       |
| RankGPT (Llama)     | -            | 8B   | 76.80 | 61.16 | 74.74  | 47.80 | 20.49 | 11.10    | 71.37 | 4.76  | 46.03 | 0.13       |
| ICR (Mistral)       | -            | 7B   | 79.49 | 67.41 | 79.50  | 52.54 | 25.65 | 10.57    | 71.44 | 4.00  | 48.83 | 0.18       |
| ICR (Llama)         | -            | 8B   | 83.93 | 69.09 | 79.96  | 53.32 | 28.36 | 10.91    | 73.10 | 4.16  | 50.35 | 0.19       |
| <b>EBCAR (ours)</b> | ConTEB Train | 126M | 76.35 | 70.99 | 72.12  | 53.88 | 31.22 | 56.92    | 76.93 | 39.57 | 59.75 | 29.33      |

Table 1: nDCG@10 across datasets. **Orange** denotes in-distribution tests, **Green** denotes out-of-distribution tests, and **Blue** denotes results of retriever-only methods. Throughput is the number of queries processed per second on a single A100 GPU.



# Recent Progress - Entity Linking

- Entity Linking is a process of determining whether two structured entities refer to the same entity.
  - Generative Linking.** Model is trained to estimate  $P(A)$ .
  - Discriminative Linking.** Model is trained to estimate  $P(\text{same} \mid A, B)$ .

$A = \{\text{Name: Jon, Age: 29}\}$   
 $B = \{\text{Name: John, Job: Intern}\}$   
 $A \cup B = \{\text{Name: [Jon, John], Age: 29, Job: Intern}\}$

$$S(A, B) = \log P(A \cup B) - \log P(A) - \log P(B) - \log(\text{Prior}).$$

Linking  
score of A  
and B

Estimated  
log-prob of  
the  
combined  
entity

Estimated  
log-prob of  
A

Estimated  
log-prob of  
B

Constant

$A = \{\text{Name: Jon, Age: 29}\}$   
 $B = \{\text{Name: John, Job: Intern}\}$

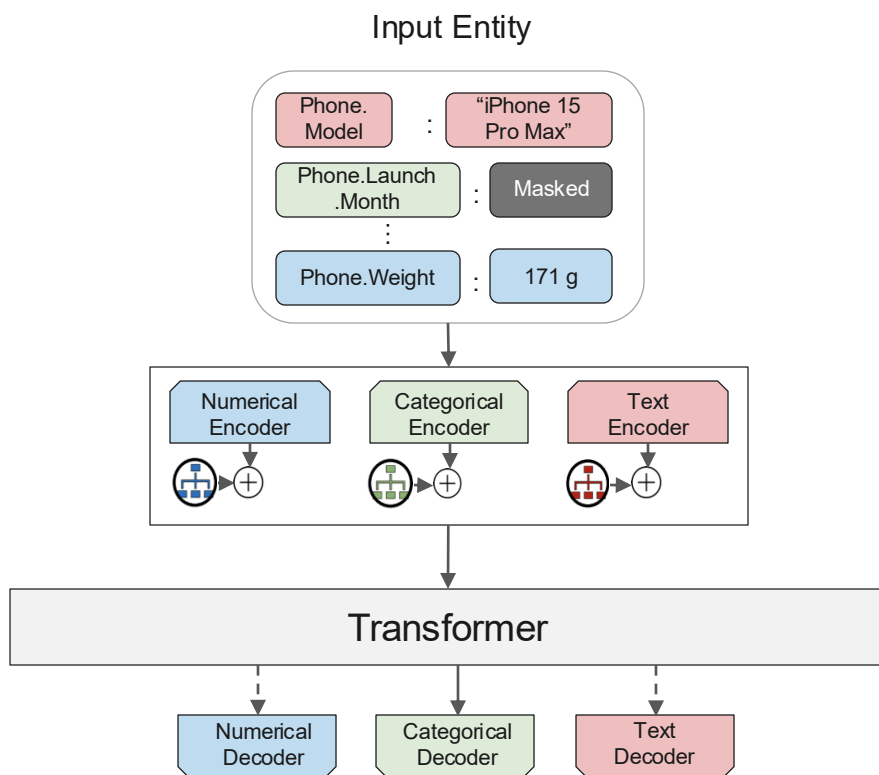
$$S(A, B) = P(\text{same} \mid A, B).$$

Linking  
score of A  
and B

Estimated prob  
according to  
the  
discriminative  
model

# Recent Progress – Entity Linking

- Generative Linking.**



- Discriminative Linking.**

